

Now, let us familiarise ourselves with a Python script that prints the parse tree of a string. First, let us try to understand what a parse tree is. A parse tree represents the syntactic structure of a sentence in a graphical format, where each node in the tree represents a part of the sentence. The tree is constructed by breaking down the sentence into its parts and grouping them into phrases based on their grammatical relationships. Parse trees are important in NLP because they help to understand the grammatical structure of a sentence. Now, consider the Python script *nltk2.py* shown below, which generates a parse tree for a given string (line numbers are added for ease of explanation).

```
1. from nltk.corpus.reader.bracket_
   parse import BracketParseCorpusReader
2. corpus_root = '/media/deepu/Data'
3. corpus =
   BracketParseCorpusReader(corpus_root,
   r'.*\..mrg')
4. tree = corpus.parsed_sents('test.
   mrg')[0]
5. tree.draw()
```

Before trying to understand how the code works, we need to discuss treebanks. According to Wikipedia, “In linguistics, a treebank is a parsed text corpus that annotates syntactic or semantic sentence structure.” Not a very simple explanation, I suppose. Well, think of treebanks as data sets that tag different words as one among the different parts of speech in English. Of course, this is an oversimplification. In English, the eight parts of speech are noun, verb, adjective, adverb, pronoun, preposition, conjunction, and interjection. If you are not familiar with these concepts, I suggest you pause and read at least the Wikipedia article titled ‘Parts of speech’. If you plan to work extensively on NLP, I suggest that you receive proper training from a linguistics expert. One of the most popular treebanks available for use

is the Penn treebank (PTB), a data set maintained by the University of Pennsylvania. PTB contains over four million annotated words. To generate parse trees, sample files from PTB can be accessed using the method ‘*treebank.parsed_sents()*’ provided by the module *nltk.corpus*. However, the sample files available in PTB are relatively large and difficult to understand. Therefore, I have manually created a simple file in PTB format (a file with extension *.mrg*). The sample file, called *test.mrg*, shown below, contains the PTB format for the sentence ‘I am Deepu’.

```
(S (NP (PRP I)) (VP (VBP am) (NP (NNP
   Deepu))))
```

The file above represents a simple tree structure that can display the given sentence as a hierarchical structure of phrases. First, the subject ‘I’ is shown as a noun phrase (NP) and the predicate ‘am Deepu’ is shown as a verb phrase (VP). Later, the next level in the hierarchy shows the word ‘I’ as a personal pronoun (PRP), the word ‘am’ as a verb in the base form (VBP) and the word ‘Deepu’ as a noun phrase (NP). Finally, in the last level, the word ‘Deepu’ is shown as a proper noun (NNP). Now, let us try to understand the Python script *nltk2.py* line by line. Line 1 imports the *BracketParseCorpusReader()* method which is used to read files in PTB format. Line 2 stores the path to the directory containing the test file in PTB format. Line 3 uses the *BracketParseCorpusReader()* method to define a reader capable of reading and processing files in formats like PTB. Line 4 reads the test file named *test.mrg* which is stored in the PTB format. Finally, Line 5 draws the parse tree for the string ‘I am Deepu’ from the file *test.mrg*. Figure 5 shows the parse tree generated by the Python script *nltk2.py*.

NLTK offers many other features for NLP. Remember that we often have to download data sets and parsers to work with NLTK. An example of a parser that can be downloaded and used while working with NLTK is the Stanford parser. A detailed discussion of parsers and context-free grammars (CFG), which are used to build parsers, is beyond the scope of our discussion. However, for extremely eager students, I suggest the popular textbook ‘Introduction to Automata Theory, Languages, and Computation’ by John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. But keep in mind that ‘Theory of Computation’ is a relatively difficult area in computer science to learn. Now, let us shift our focus to data science.

Introduction to PySpark

If you recall the Venn diagram from the first article in this series, you will know that data science intersects with AI and machine learning. Data science also includes sub-fields like Big Data analytics, which usually doesn’t involve AI or machine learning. This section focuses on introducing the development of NLP-based data science applications using PySpark, an interface for Apache Spark in Python. Apache Spark is an open source analytics engine for large scale data processing. It is written in Scala and licensed under Apache License version 2.0.

Let us now talk about the significance of PySpark, which is powered by machine learning. It has three main modules: *pyspark.sql* for structured data and DataFrames, *pyspark.ml* for high-level API machine learning models, and *pyspark.mllib* for low-level API machine learning models. While Apache Hadoop is a widely-used framework for Big Data analytics, it lacks AI and machine learning capabilities. However, PySpark